

Eli Safety Parameter Guidebook

Complete reference for every Tier 1 safety parameter implemented in the Eli API system. Covers authentication, rate limiting, payload limits, input sanitization, prompt injection blocking, key validation, and audit logging for a limited-key LLM architecture.

Author: Z.ai · **Date:** August 2026

Version: Tier 1 · **Scope:** 4 Routes, 9 Chapters

Table of Contents

h_9ce0121b	3
h_3fffbef9	3
h_18c23cd9	3
h_56dc63b7	4
h_09d29482	4
h_92f53fa3	4
h_66e734ac	4
h_32d1381b	5
h_2bab4409	5
h_58bf8387	5
h_4485a8b3	6
h_bb36a821	6
h_fcae09f0	6
h_f7c8101b	6
h_86e08d1f	7
h_6e7676f0	7
h_0eb9e36f	7
h_7a10be65	7
h_10a6fef9	7

h_10281776	8
h_2a59b766	8
h_e71f3d30	8
h_0997f09d	8
h_20fb1209	9
h_18317c9a	10

Chapter 1: Safety Architecture Overview

1.1 The Limited Key Model

Eli operates on a fundamentally constrained resource model: a small, fixed pool of API keys managed by the OmniRoute rotation coordinator. Unlike traditional SaaS applications that can provision unlimited API credentials, Eli relies on a combination of long-lived Gemini keys (such as the primary AQ.-prefixed key) and temporary disposable keys generated by the OpenClaw email engine. This constraint shapes every safety decision in the system. When your key pool is finite and partially auto-generated, every request that reaches an LLM API consumes a scarce resource that cannot be arbitrarily replenished.

The OmniRoute coordinator maintains a penalty-tracked pool where each key carries a health score. Keys that return 429 rate-limit errors, 5xx server errors, or timeout failures accumulate penalty points. Once a key reaches the penalty ceiling, it is rotated out and the next healthiest key takes over. An attacker who attempts to drain keys through rapid requests triggers automatic rotation, but rotation itself consumes inboxes and provider capacity, which are also finite. The safety system must protect not just the active key but the entire key lifecycle.

OpenClaw generates temporary email inboxes across three providers: Guerrilla Mail, mail.tm, and OpenInbox. Each inbox has a 55-minute TTL, can hold at most one API key, and the system maintains a maximum of 10 concurrent inboxes with a hard cap of 20 pending keys. These numbers represent real resource constraints. A sustained attack forcing rapid rotation could exhaust email provider limits, leaving Eli without recovery paths.

1.2 Three-Tier Safety Model

The safety system is organized into three progressive tiers. Tier 1, the focus of this guidebook, provides essential guards that prevent immediate abuse: authentication, input validation, rate limiting, payload limits, key format validation, prompt injection blocking, and structured audit logging. These are the minimum viable safety parameters any API-facing system should implement.

Tier	Scope	Key Mechanisms	Status
Tier 1	API gate, input, output	Auth, rate limit, payload, sanitization, key validation, audit	Implemented
Tier 2	Session, identity, quota	Per-session keys, scoped rate limits, user identity, call gating	Planned
Tier 3	Resource, tenant, expiry	Per-resource permissions, tenant isolation, key expiry, rotation schedules	Future

Table 1: Three-tier safety model with implementation status.

Tier 2 will introduce per-session API keys generated for each authenticated user, eliminating the need to share the master key. The capability level system (public, user, operator, admin) will become fully enforced with

per-session capability tokens. Tier 3 adds resource-level permissions, tenant isolation for multi-organization deployments, and automated key expiry policies.

1.3 Centralized Safety Module

All safety parameters live in a single source-of-truth module: `src/lib/safety-gate.ts`. Every constant, function, and type that route handlers need is exported from this file with JSDoc documentation explaining what it is, why it exists, and where it is used. No route file contains hardcoded safety values. This centralization means auditing the entire safety surface requires reading one file, and changing a limit requires editing exactly one place.

The audit log (`src/lib/audit-log.ts`) provides a structured, persistent record of every security-relevant event. Each entry includes an ISO timestamp, event type, detail, metadata, and caller IP. Events are buffered in memory (capped at 500 entries) and asynchronously appended to a JSONL file on disk. This is the forensic record that makes post-incident analysis possible.

Chapter 2: API Authentication Gate

2.1 Design Philosophy

The authentication gate is the outermost defense layer. In Tier 1, it uses a single shared bearer token checked against the `ELI_API_KEY` environment variable. This is intentionally simple: the Tier 1 threat model targets casual discovery and automated scanning. Someone who discovers the API URL should be met with a 401 response, not an open endpoint that drains LLM keys. The gate supports two methods: the standard `Authorization: Bearer` header, and a `?key=` query parameter for convenient curl testing.

2.2 Defense-in-Depth Check Ordering

The `checkAuth()` function accepts any object with `headers` and optional `url` properties, making it compatible with Next.js without framework coupling. Every route handler calls this first, before rate limiting, before payload parsing, before business logic. This ordering ensures a blocked request consumes zero server resources beyond the authentication check itself.

Step	Check	Fail Response	Cost
1	checkAuth(request)	401 Unauthorized	Near-zero (string compare)
2	checkRateLimit(ip, config)	429 Too Many Requests	Map lookup + array push
3	Payload size (content-length)	413 Payload Too Large	Integer parse + compare
4	Input validation / sanitization	400 Bad Request	Regex + string ops
5	Business logic (LLM call)	Varies	High (network I/O)

Table 2: Defense-in-depth check ordering from cheapest to most expensive.

2.3 Capability Gating

The `checkCapability()` function adds a capability level system on top of auth. Four levels are defined: public (no auth, for health checks), user (read-only state), operator (modify non-sensitive state like rotating keys), and admin (full access including raw keys and audit logs). In Tier 1, all authenticated users are treated as admin since there is one master key. The `OMNI_CAPABILITIES` array maps every omni route action to its required level, making the permission model explicit and auditable for Tier 2 implementation.

Level	Hierarchy	Access Scope	Example Actions
public	0	No auth required	Health check, GET /api/health
user	1	Read-only system state	View omni state (masked keys), record usage
operator	2	Modify non-sensitive state	Force rotation, create inbox, reject pending key
admin	3	Full access including secrets	Read raw key, inject key, approve, read audit log

Table 3: Capability level hierarchy and access scope.

Chapter 3: Rate Limiting

3.1 Why Rate Limiting Is Critical

Rate limiting in Eli serves a dual purpose: preventing server overload and protecting finite LLM API key quotas. A single Gemini free-tier key might allow 15 requests per minute and 1,500 per day. An attacker sending 100 requests per second would drain the daily quota in 15 seconds. The rate limiter acts as a resource preservation tool, not just a performance tool. The implementation uses an in-memory sliding window algorithm: each IP address maintains a list of request timestamps, pruned on each request to remove entries outside the configured window duration.

3.2 Per-Route Configuration

Route	Limit	Window	Rationale
/api/eli-chat POST	15 req/min	60s	LLM key budget; Gemini free tier bottleneck
/api/omni GET	30 req/min	60s	Monitoring reads are cheap; higher for dashboards
/api/omni POST	5 req/min	60s	Write actions spawn inboxes, call providers
/api/vault-sync GET	20 req/min	60s	File I/O bounded; prevent scraping 24K chunks
/api/audit GET	10 req/min	60s	Admin endpoint; low tolerance for info disclosure
/api/health GET	120 req/min	60s	Monitoring pings; very permissive

Table 4: Per-route rate limit configuration with rationale.

3.3 Sliding Window Algorithm

The `checkRateLimit(ip, config)` function maintains a global Map of IP-to-timestamp-array entries. Every 5 minutes, stale entries are pruned to prevent unbounded memory growth. The function returns true if the request is allowed, false if rejected. Crucially, rejected requests do not consume a slot; only allowed requests are recorded. The `getRateLimitState()` function provides a debug view showing request counts per IP for verification during testing.

Chapter 4: Payload Size Limits

4.1 Per-Route Payload Caps

Payload limits prevent memory exhaustion attacks and oversized payloads that could crash the server during JSON parsing. Each route has a specific limit tuned to its expected input size. The absolute maximum across all routes is 100KB, a hard ceiling that no route may exceed regardless of its individual limit. The limits are checked before the request body is parsed, using the Content-Length header, making the check nearly free. If Content-Length is missing or zero, the body is still parsed but the payload limit serves as a secondary defense.

Route	Limit	Why This Size
eli-chat POST	10 KB	Chat messages are short; LLM context is the bottleneck, not bandwidth
omni POST	10 KB	Action bodies are small JSON (action name + optional key/inboxId)
vault-sync GET	4 KB	Query params only; no body expected
Absolute ceiling	100 KB	No route may accept more, even if individually configured higher

Table 5: Payload size limits per route with sizing rationale.

Chapter 5: Input Sanitization

5.1 sanitizeInput(): Text Normalization

The `sanitizeInput()` function processes all user-supplied text before it reaches the LLM or gets stored. It performs five operations: (1) removes null bytes that could truncate strings or cause downstream parsing errors; (2) normalizes Unicode to NFC canonical form, preventing homoglyph attacks where different Unicode representations of the same character bypass filters; (3) collapses excessive whitespace (more than 3 consecutive newlines or spaces); (4) removes non-printable control characters except newline, carriage return, and tab; (5) trims and enforces a maximum length (default 4000 characters). This does not HTML-escape content, as React handles XSS protection on the frontend.

5.2 sanitizePromptInjection(): LLM Attack Prevention

Prompt injection detection uses 12 regex patterns covering the most common attack vectors: system prompt override attempts ("ignore previous instructions"), delimiter injection ("---END SYSTEM---"), role hijacking ("you are now..."), output format manipulation ("respond only with JSON"), and known jailbreak markers ("DAN mode", "jailbreak"). In the current implementation, detection triggers an immediate 400 rejection with an `injectionDetected: true` flag. This is a deliberate choice: flag-only mode (where the message passes through but is logged) was the previous behavior, but it allowed potential injection attempts to reach the LLM. The blocking approach is defense-in-depth: even though the system prompt is designed to resist injection, there is no reason to give an attacker a free attempt.

Chapter 6: Key Validation

6.1 Format Validation Patterns

Every API key must pass format validation before it enters the key pool. The `validateKeyFormat(service, key)` function checks the key against a service-specific regex pattern, a minimum length, and a maximum length (500 characters). Keys that fail format validation are rejected at the injection point and logged to the audit trail. This prevents malformed keys from causing API errors, wasting rotation attempts, or being used as injection vectors against the target API service.

Service	Pattern	Min Length	Example
gemini	AIza... or AQ....	20 chars	AIzaSyD... or AQ.Ab8RN6...
openai	sk-...	25 chars	sk-proj-abc...
anthropic	sk-ant-...	30 chars	sk-ant-api03-...

Table 6: Key format validation patterns by service.

6.2 Approval Queue

Keys extracted by OpenClaw from email do not automatically enter the active key pool. They enter a pending queue with a maximum capacity of 20 entries. By default, auto-approve is OFF, meaning each key must be manually approved via `POST /api/omni?action=approve` before it becomes active. The approval process includes a format check and a live API test call to the target service (e.g., a simple "Say OK" request to Gemini). Only keys that pass both checks are promoted to the active pool. This two-step validation (format + live test) catches keys that look valid but are actually revoked, rate-limited, or belong to a different project.

Chapter 7: Audit Logging

7.1 Event Types and Structure

The audit log records 14 event types covering the full lifecycle of security-relevant operations. Each entry is an object with an ISO timestamp, event type string, human-readable detail, optional structured metadata, and optional caller IP. Events are buffered in memory (capped at 500 entries by the `MAX_AUDIT_MEMORY` constant) and asynchronously appended to a JSONL file at `data/audit/audit.jsonl`. The async write ensures the audit call never blocks the request handler. The `/api/audit` endpoint (admin-only, rate-limited to 10 req/min) provides read access to the in-memory buffer.

Event Type	Description	Typical Metadata
auth.blocked	Authentication failed	ip, action
chat.ratelimited	Chat rate limit exceeded	ip
prompt.injection.blocked	Prompt injection detected	ip, messagePreview
key.extracted	OpenClaw pulled key from email	service, inboxEmail
key.injected	Manual key injection	service, keyPreview
key.rotation	Key rotation occurred	service, inboxEmail
llm.call	LLM API called	ip
llm.failure	LLM API call failed	ip, error
vault.query	Vault search executed	query, resultCount

Table 7: Core audit event types with descriptions and metadata.

Chapter 8: Route-by-Route Safety Wiring

This chapter documents exactly which safety parameters are applied in each route handler, in execution order. Every route follows the same pattern: auth check, rate limit, payload check, input validation, then business logic. The table below summarizes the complete wiring across all four protected routes.

Route	Auth	Rate Limit	Payload	Input Sanitization	Prompt Injection
/api/eli-chat POST	checkAuth()	15/min	10KB	sanitizeInput()	BLOCK + 400
/api/omni GET	checkCapability()	30/min	N/A (GET)	N/A	N/A
/api/omni POST	checkAuth()	5/min	10KB	validateKeyFormat()	N/A
/api/vault-sync GET	checkAuth()	20/min	N/A (GET)	N/A	N/A
/api/audit GET	checkAuth()	10/min	N/A (GET)	N/A	N/A
/api/health GET	None	120/min	N/A	N/A	N/A

Table 8: Complete safety wiring matrix across all routes.

Note that /api/health is the only route without auth. This is intentional: health checks are used by monitoring systems and load balancers that cannot carry authentication credentials. The 120 req/min rate limit provides sufficient protection against abuse while allowing aggressive monitoring. If this becomes a concern, the health endpoint can be restricted to internal network ranges at the Caddy level.

Chapter 9: Configuration Reference

This chapter provides the complete configuration reference for all safety parameters. Every constant defined in safety-gate.ts is listed with its current value, type, and the rationale for why it is set to that specific value. This reference is intended for anyone who needs to tune the safety parameters for a different deployment environment or threat model.

Parameter	Value	Type	Rationale
MAX_PAYLOAD_CHAT	10,240 bytes	int	Chat messages are short; prevents OOM in JSON parsing
MAX_PAYLOAD_OMNI	10,240 bytes	int	Omni actions are small JSON payloads
MAX_PAYLOAD_ABSOLUTE	100,000 bytes	int	Hard ceiling no route may exceed
RATE_LIMIT_CHAT	15 req / 60s	object	Matches Gemini free-tier rate budget
RATE_LIMIT_OMNI_POST	5 req / 60s	object	Write ops are expensive (rotation, injection)
MAX_HISTORY_MESSAGES	20	int	LLM context window is limited; recent context only
MAX_MESSAGE_LENGTH	4,000 chars	int	Prevents token budget exhaustion from single message
MAX_PENDING_KEYS	20	int	Approval queue cap; oldest auto-rejected when full
MAX_CLAW_INBOXES	10	int	Provider rate limit preservation
MAX_AUDIT_MEMORY	500	int	Memory cap for in-memory audit buffer
INBOX_TTL_MS	55 minutes	int	Provider TTL minus 5min safety margin

Table 9: Complete safety parameter reference.

9.1 Environment Variables

Variable	Required	Description
ELI_API_KEY	No (enables auth)	Master bearer token. When set, all routes (except /health) require it
GEMINI_API_KEY	Yes	Primary Gemini API key. Can be AIza... or AQ... format

Table 10: Environment variables affecting safety behavior.

When `ELI_API_KEY` is not set, the authentication gate is disabled and all requests pass through. This is the correct behavior for local development but must never be the case in production. The deployment script does not set this variable; it must be configured on the server via systemd environment file or `.env`.